

6-Axis Motion Sensor (BMI160)

Hardware Overview

What is it?

The **BMI160** is a "6-axis" Inertial Measurement Unit (IMU). It contains two sensors in one tiny package: a 3-axis Accelerometer (gravity and tilt) and a 3-axis Gyroscope (rotation speed). It allows you to detect if the device is being tilted, shaken, or spun and to get X, Y and Z angle.

The **NEO-6M GPS Module** is a GPS-receiver that picks up radio signals from satellites orbiting the Earth. It gives you the exact Location (Latitude/Longitude), Altitude, Speed, and time.

Electrical Explanation

- **Accelerometer:** Measures the force of gravity ($9,81\text{m/s}^2$). By seeing which direction gravity is pulling, the chip calculates **Pitch** and **Roll**.
- **Gyroscope:** Measures how fast the device is spinning in degrees per second. Unlike the accelerometer, it doesn't care about gravity; it only feels movement.
- **Communication:** It uses the **I²C protocol** and shares **SDA** and **SCL** wires with the display.

Key Hardware Facts

- **Connection:** I²C Bus (SDA/SCL).
- **Default Address:** 0x69.
- **Logic Voltage:** 5V
- **Signal Flow:** Motion → BMI160 Chip → Analog Voltage/ADC → I2C Data → Processor.

Software & Programming

kcomp_GYBMI160.h

This library handles the complex trigonometry required to turn raw G-forces into degrees of tilt.

Usage: `#include <kcomp_GYBMI160.h>`

The Data Structure

When you call `readBMI160()`, you receive a `BMI160Data` struct containing:

Field	Type	Description
accelX/Y/Z	float	Raw acceleration in m/s^2 .
gyroX/Y/Z	float	Raw rotation speed in deg/s .
angleX	float	Pitch: Forward/Backward tilt in degrees.
angleY	float	Roll: Left/Right tilt in degrees.
angleZ	float	Yaw: Left/Right rotation (heading) in degrees.

Function Reference

Function Name	Description & Parameters
---------------	--------------------------

void initBMI160()	Setup: Initializes the sensor. Must be called in <code>setup()</code> . Returns <code>true</code> if the sensor was initialized successfully.
BMI160Data readBMI160()	Read: Returns the latest motion and angle data.
void resetGyroZ()	Reset: Sets the z-angle (Yaw) back to zero.

⚠ Pitfalls & Troubleshooting

There is a major difference between how the library calculates X/Y and Z angles:

- Pitch & Roll (x and y): These use gravity as "anchor." They are very stable and never drift.
- Yaw (z): Gravity doesn't change when you spin flat on a table. Therefore, the library must estimate the z-angle by adding up gyro speeds over time (by numerical integration).

The Consequence: Because of tiny mathematical errors that add up, the z-angle will slowly "drift" even if the sensor is perfectly still. You should provide a button in your code to call `resetGyroZ()` occasionally. Furthermore, you have to call `readBMI160()` at least every 50ms to get acceptable z-angle readings.

Sample Program

This program displays your location.

```
#include "kcomp_GYBMI160.h"

void setup() {
    Serial.begin(115200);

    // --- THE INITIALIZATION CHECK ---
    // initBMI160() returns 'true' if the Chip ID (0x39) is found.
    // We stop the program if the sensor is missing.
    if (!initBMI160()) {
        Serial.println("CRITICAL ERROR: BMI160 sensor not found!");
        Serial.println("1. Check SDA/SCL wiring.");
        Serial.println("2. Check if the module has power (VCC/GND).");
        // Stop the program here (infinite loop).
        while(1);
    }

    Serial.println("Motion Sensor Ready!");
}

void loop() {
    // 1. Get the latest data
    BMI160Data data = readBMI160();

    // 2. Print the angles
    Serial.print("Pitch: "); Serial.print(data.angleX);
    Serial.print(" | Roll: "); Serial.println(data.angleY);

    // 3. Logic Example: Tilt-to-Action
    if (data.angleX > 30) { Serial.println("Action: MOVING FORWARD"); }
    else if (data.angleX < -30) { Serial.println("Action: BRAKING"); }

    // IMPORTANT: For accurate Z-angle (Yaw), keep the loop fast!
    delay(50);
}
```

 **Try it out:** You can find the full sample program **SimpleGYBMI160** in the K-Comp examples folder!